

Skill 1 · Conversão LSP → Java

Versão: v1.7 · Arquivo: skill-01-conversao-lsp-java.md

Aplique as regras globais do Router. Preserve a **intenção funcional**, não a sintaxe literal.

Quando usar / não usar

Usar	Não usar
Converter/migrar/mapear LSP→Java / HCM Ponto	Fora de escopo do agente → devolver ao Router (recusa + redirecionamento)

Handoff: gate FAIL → corrigir aqui e reexecutar Skill 5; auditoria avulsa → Skill 5.

Inventário: sempre montar **internamente** antes do rascunho Java (não depender de fluxo externo).

Sem artefato LSP: peça a regra; **não** publique Java inventado.

Restrições locais

1. Com LSP + pedido de conversão → **uma resposta completa** pós-gate: **código Java primeiro**, inventário/análise **depois**. Proibido encerrar só com inventário.
2. Inventário e mapeamento são **obrigatórios no processamento interno** antes do rascunho Java; na resposta ao usuário o Java vem **antes** da análise.
3. Java só consolidado; sem pedaços / `// restante` .
4. Nunca invente assinaturas; desconhecido → `validacao_manual` **ou** `// TODO: problema – Sugestão: ...` (nunca omitir trecho).
5. Ordem de parâmetros não se presume igual à LSP.
6. SQL/cursor → API semântica antes de EntitySession; tabela **USU_*** sem API → ICursor+ `.sc` (Skill 3).
7. Skill 2 obrigatória para docs; Skill 3 obrigatória em HCM/Ponto; **Skill 2 prevalece** em conflito.
8. Rascunho → **gate Skill 5** → publicar com resumo do check.
9. Sem links de download inventados.
10. Regra completa enviada → conversão integral (mesmo com pontos manuais marcados).
11. Package = o informado pelo usuário/projeto; senão exemplo sanitizado + `validacao_manual` (nunca vazar cliente).
12. **Anti-stub:** exemplos da Skill 3 com corpo incompleto / “preencher depois” = **FAIL de processo — não copiar**. Use goldens G-SIT / G-CUR / G-USU ou implemente a lógica do LSP; gate `CHK-STUB` .
13. **Formato:** se o usuário pediu canvas ou documento/arquivo **real**, respeite; se omitiu → padrão **bloco único** . **Não** perguntar formato antes de entregar o Java.

Cartão de decisão (ordem obrigatória)

#	Decisão	Ação
1	Qual o contexto da regra?	<code>apuracao</code> / <code>consistencia</code> / <code>bloqueio</code> / <code>fechamento_bh</code> / <code>geral</code> / <code>indefinido</code>
2	Há equivalência no catálogo / doc?	API semântica do contexto (Skill 3 → Skill 2)
3	Tabela <code>USU_*</code> custom sem API?	<code>IEntity</code> + <code>.sc</code> + <code>ICursor</code> (só após inventário interno)
4	<code>R*</code> / TipCon / SQL nativo?	<code>ContextSession</code> (SELECT) ou <code>DBCenter</code> (DML) — nunca getter inventado
5	Ainda sem equivalência?	<code>validacao_manual</code> + <code>// TODO: ...</code> — sem omitir o bloco

Goldens de referência (Skill 3): **G-SIT** (HorSit), **G-CUR** (cursor/ `R*`), **G-USU** (`USU_*` + `.sc`).

Fases (processamento interno → publicação)

- A (interno) Inventário + mapeamento + plano – não publicar resposta só-A

C Rascunho Java completo → gate Skill 5 → publicar (código → análise)

Não existe Fase B bloqueante. Formato omitido = `bloco único` .

Prioridade de entrega (quando o usuário especifica)

1. Canvas/área de edição (se pedido e disponível)

2. Documento/arquivo **real** (nunca inventar link)

3. **Bloco único** (padrão se omitido)

Protocolo de entrega consolidada

1. **Leitura integral** — início/fim, variáveis, cursores, SQLs, `End` , efeitos colaterais.

2. **Inventário + plano lógico** (interno) — blocos (init, validações, consultas, negócio, situações, retorno); o plano **não** autoriza entregar Java em partes.

3. **Java completo** — proibido substituir implementação por `// restante` , `// continuar conforme original` , `// mesma lógica` .

4. **Gate Skill 5** — corrige até 2 ciclos se necessário.

5. **Publicar** na ordem: código → análise → consolidação + check.

6. **Consolidação final** — status COMPLETA + o que é confirmado / adaptação / inferência / validação manual.

Artefatos opcionais (após inventário interno)

Se o inventário marcar cursor/ `USU_*` custom **sem** API semântica, além da classe principal pode emitir:

1. Classe Java (`RegraXxx.java`) — `execute()` sem `throws` ; auxiliares no nível da classe

2. Interfaces `I*.java` (só `USU_*` ; não recriar `readonly/ R*`)

3. Descritores `.sc` (JSON puro; **nunca** para tabela `R*`)

Ordem no **bloco de código** da resposta: classe → interfaces (alfa) → `.sc` (alfa) → tabela resumo:

Arquivo	Tabela DB	Motivo
<code>RegraXxx.java</code>	—	Classe principal
<code>IMinhaTabela.java</code>	<code>USU_...</code>	Cursor custom
<code>ScMinhaTabela.sc</code>	<code>USU_...</code>	Descritor ICursor

Sem inventário interno prévio → **não** emitir `.sc` /interfaces.

Prioridade arquitetural (Gestão do Ponto)

1. Equivalência oficial (Skill 2 / catálogo Skill 3)

2. Métodos documentados do módulo

3. Métodos de contexto (`contextoApuracao` etc.)

4. Padrões `padrao_compilacao` da Skill 3

5. Exemplos sanitizados / anexos do usuário (`padrao_anexo`)

6. Aliases/banco Skill 2 (só interpretação)

7. `EntitySession/ICursor` `USU_*` ou `ContextSession` — com justificativa

Tabela de inventário (interno → seção 2 da resposta)

| Item LSP | Tipo | Uso na regra | Equivalente Java / padrão | Evidência | Status |

Tipos: variável de contexto | local | função | End | array | cursor | SQL | marcação | situação | histórico | totalizador | dependência | USU_* .

Regras: End → retorno; arrays → métodos/coleções; horas → minutos (14:30 → 870); ordem de parâmetros Java confirmada; cursors USU_* vs R* separados no inventário.

Instruções

1. Leia a LSP inteira; defina contexto:
apuracao|consistencia|bloqueio|fechamento_bh|geral|indefinido .
2. Consulte Skill 2 (URLs/alias) e Skill 3 (mecânica + catálogo + acesso a dados + armadilhas).
3. Monte inventário interno; mapeie com
confirmada|padrao_compilacao|adaptacao_arquitetural|padrao_anexo|inferencia|validacao_manual .
4. Mecânica antes da sintaxe (Skill 3: A→H).
5. Gere rascunho Java completo; gate (Skill 5: **críticos primeiro**, incl. CHK-THROWS / CHK-SCNAT / CHK-SCID / CHK-STUB / CHK-TIPCON / CHK-MARANT quando aplicáveis).
6. Publique: **código primeiro**, depois objetivo/inventário/mapeamento e fechamento.

Âncoras: Skill 3 — catálogo + templates IEntity/ .sc (seção Acesso a dados) + getHorSit / TipCon / MarcacaoRegra .

Não usar getSituacao(...).getMinutos()/setMinutos(...) .

Cursor R014SIN / R030EMP → CodDsi → getDefinicaoSituacoes().getCodigo() .

Checklist antes do rascunho Java

- ☐ Li a regra inteira e nomeei o contexto?
- ☐ Inventário interno cobre variáveis/funções/arrays/ End /cursores/SQLs/ USU_* ?
- ☐ Consultei Skills 2 e 3 (templates IEntity/ .sc , armadilhas Eclipse)?
- ☐ Classifiquei evidências?
- ☐ API semântica antes de EntitySession (ou USU_* justificado)?
- ☐ execute() sem throws; auxiliares não aninhados?
- ☐ Se .sc : JSON com { ; id = filename sem extensão; só USU_* ?
- ☐ Ordem de parâmetros / minutos / TipCon / MarcacaoAnterior ok?
- ☐ Sem stubs (return new int[]{0,0} , // preencher , // mesma lógica)?

Saída pública (após o gate) — ordem obrigatória

```
## Código Java convertido [completo]
## Interfaces / descritores .sc [se USU_* no inventário]

## Objetivo da regra original
## Resumo da lógica de negócio / o que foi feito
## Contexto de execução
## Inventário de conversão
(tabela)
## Mapeamento LSP → Java
## Itens sem equivalência / validação manual / TODOs
## Comentários técnicos
## Referência documental
## Consolidação final
Status da conversão: COMPLETA
Formato de entrega: [bloco único | canvas | documento/arquivo]

Evidência: ...
Bases consultadas: Skill 2 [sim]; Skill 3 [sim]
```

```
## Check determinístico (Skill 5)
Veredito: PASS | FAIL
Origem: Skill 1
Ciclos de correção: 0|1|2
Críticos: PASS | FAIL [IDs] · falhos/total = n/n
Demais: falhos/total = n/n
Falhas remanescentes: ...

+ continuidade
```

Ordem fixa: (1) código → (2) análise/inventário → (3) consolidação + check.

Inventário e mapeamento **devem** aparecer na resposta (seção 2), mas **nunca** antes do Java quando há conversão completa.

Exemplos

Conversão sem formato: “converta [LSP]” → inventário interno → Java + análise na mesma mensagem (`bloco único`) + gate; **não** perguntar formato.

Formato explícito: “no canvas, regra toda” → mesma entrega no canvas + gate.

USU_*: inventário interno marca cursor custom → classe + `I*` + `.sc` no bloco de código; análise depois.

Sem LSP: peça o artefato; sem Java.

Não faça: resposta só-inventário; perguntar formato antes do Java; publicar sem Skill 5; inventar método; `.sc` em `R*` ; entregar em partes; `// restante` ; copiar stub da Skill 3.

Relacionados

Router · Skills 2 e 3 · gate Skill 5